

THE ISAAC ORACLE

Step-by-Step Project Plan

Item Explorer • Synergy Graph • Build Analyzer • Data Explorer • AI Coach

Project idea

A fan-made application for The Binding of Isaac where users can search all items. Documented synergies are displayed on detail pages, as an interactive graph, and within complete builds. A data-grounded AI Coach may later explain the detected interactions.

Prepared for: Lubna Ishaq

Working method: small, testable steps with a Git commit after every stable milestone

1. Product Vision and Scope

The project will not be a copy of an existing synergy website. Its focus is a combined application with visual navigation, structured data, build analysis, and an explanatory AI feature at a later stage.

Module	Purpose
Item Explorer	All items, search, filters, images, and detail pages.
Synergy Explorer	Documented positive, dangerous, mixed, and override interactions.
Interactive Graph	Clickable item nodes and color-coded relationships.
Build Analyzer	Select multiple items and find known interactions within the build.
Data Explorer	Analysis of items, qualities, synergy types, and graph hubs.
AI Coach	Explains verified structured data and marks unknown relationships.
Scope rule Every item may appear in the Item Explorer. A synergy relationship is added only when a noteworthy interaction has been documented and reviewed. A missing synergy does not automatically mean that two items have no combined effect.	

2. Current Project Status

- ☒ GitHub repository and Codespace created
- ☒ Vite React frontend is running
- ☒ items.json created and loaded into React
- ☒ useState used for item selection
- ☒ synergies.json created
- ☒ React Router installed
- ☒ Initial synergy filter logic implemented
- ☐ Finish the HomePage and ItemPage structure
- ☐ Finalize the data model for multi-item synergies
- ☐ Add images and scale the data set

3. Working Method for Every Feature

Repeat this process for every small feature. This keeps the application stable and makes errors easier to trace.

1. Define one mini-goal

Choose one clearly testable result, such as “Clicking Godhead opens its detail page.” Do not start a second feature at the same time.

2. Check the data

Confirm that items.json or synergies.json contains the required fields.

3. Implement a small change

Change only the affected component. Keep code comments in English.

4. Test in the browser

Test the normal case, empty data, an unknown item, and backward navigation.

5. Fix errors immediately

Read the first specific error message. Do not change several files blindly.

6. Commit the stable result

After the feature works, run git add, git commit, and git push.

7. Update project notes

Record what works, what remains open, and which architecture decision was made.

Git routine

```
git status → git add . → git commit -m "Short description" → git push
```

4. Development Roadmap

Phase 1: Stable Foundation and Navigation

Goal: Turn the current single-page prototype into a stable application with a home page and a dynamic item detail page.

Work steps

- ☐ Create src/pages/HomePage.jsx
- ☐ Create src/pages/ItemPage.jsx
- ☐ Define routes in App.jsx
- ☐ Verify BrowserRouter in main.jsx
- ☐ Render items as links on the home page
- ☐ Test the /item/:itemId route
- ☐ Add an item-not-found state

Definition of Done

- ☐ Home page loads without errors
- ☐ Godhead opens /item/godhead

- ☐ Back link works
- ☐ An unknown URL displays a clear message

Result: One dynamic ItemPage renders every item, so individual pages do not need to be coded manually.

Phase 2: Long-Term Data Model

Goal: Structure items and synergies so combinations containing more than two items can be represented.

Work steps

- ☐ Add description, quality, type, and image to items.json
- ☐ Change synergies.json to use an items array
- ☐ Give every synergy a unique id
- ☐ Add type, description, gameVersion, and verified
- ☐ Handle empty or unknown references safely

Definition of Done

- ☐ Every synergy references existing item IDs
- ☐ One two-item and one three-item synergy display correctly
- ☐ Unverified records are visibly marked

Result: A flexible JSON model shared by item pages, graph, build analyzer, and AI Coach.

Phase 3: Item Explorer

Goal: Make every item easy to find and open.

Work steps

- ☐ Create a search field with useState
- ☐ Filter item names case-insensitively
- ☐ Add a quality filter
- ☐ Optionally filter by item type
- ☐ Display an empty search state
- ☐ Make item cards responsive

Definition of Done

- ☐ Partial-name search works
- ☐ Filters can be reset
- ☐ No-results state does not crash
- ☐ Cards remain usable on a narrow screen

Result: A usable home page for a large item data set.

Phase 4: Images and Media

Goal: Add structured item images and, later, synergy screenshots or GIFs.

Work steps

- ☐ Create public/images/items

- ☐ Use the item ID as the file name
- ☐ Store the image path in items.json
- ☐ Add a fallback image
- ☐ Create public/images/synergies
- ☐ Support an optional media object
- ☐ Document the source and permitted use of every asset

Definition of Done

- ☐ Missing images do not break the layout
- ☐ Item cards show image and name
- ☐ Synergy media appears only when available

Result: A visual Item Explorer with reliable media fallbacks.

Phase 5: Synergy Detail View

Goal: Display every documented relationship clearly on each item page.

Work steps

- ☐ Filter with `synergy.items.includes(itemId)`
- ☐ Determine all related item IDs
- ☐ Resolve IDs to readable item names
- ☐ Color cards by synergy type
- ☐ Show description, version, and verification status
- ☐ Link cards to related item pages

Definition of Done

- ☐ Items without synergies have a clear empty state
- ☐ Multi-item synergies list all partners
- ☐ Every related item is clickable

Result: A complete and navigable Synergy Explorer.

Phase 6: Interactive Synergy Graph

Goal: Visualize the existing relationships as an interactive network.

Work steps

- ☐ Use React Flow in a separate graph component
- ☐ Generate one node per item
- ☐ Generate edges from documented relationships
- ☐ Use colors for positive, danger, mixed, and override
- ☐ Focus the selected item
- ☐ Navigate to an item page on node click
- ☐ Add zoom, fit view, and a legend

Definition of Done

- ☐ Graph loads from JSON instead of hardcoded nodes
- ☐ No duplicate edges

- ☐ Click navigation works
- ☐ Large graphs remain usable

Result: A visual Synergy Graph that becomes the project's signature feature.

Phase 7: Build Analyzer

Goal: Select multiple items and detect known relationships inside the build.

Work steps

- ☐ Store selected build items in an array
- ☐ Add and remove items
- ☐ Prevent duplicate selections
- ☐ Find synergies whose required items are present
- ☐ Separate positive, dangerous, and override interactions
- ☐ Treat unknown relationships as unknown
- ☐ Allow a build to be exported by URL or JSON

Definition of Done

- ☐ A test build detects the expected relationships
- ☐ Removing one item updates results immediately
- ☐ The application does not invent a rating

Result: A transparent Build Analyzer based on documented rules.

Phase 8: Data Explorer

Goal: Create meaningful metrics and visualizations from the data set.

Work steps

- ☐ Calculate synergy count per item
- ☐ Show highly connected items
- ☐ Visualize quality distribution
- ☐ Visualize synergy-type distribution
- ☐ Count verified and unverified records
- ☐ Expose broken references and data issues

Definition of Done

- ☐ Metrics are calculated from JSON
- ☐ Charts react to data changes
- ☐ Values are not manually entered

Result: A data-science-oriented analysis section for the portfolio.

Phase 9: Data Import and Quality Assurance

Goal: Replace the manual test data with a larger controlled data set.

Work steps

- ☐ Review allowed data sources and licenses
- ☐ Write imported content to a separate raw file

- ☐ Normalize names and IDs
- ☐ Detect duplicates
- ☐ Handle images separately
- ☐ Create validation for required fields
- ☐ Review a representative sample before publishing

Definition of Done

- ☐ Importer does not overwrite curated data accidentally
- ☐ Validation reports broken references
- ☐ The source is documented for imported records

Result: A reproducible and reviewable import process instead of blind scraping.

Phase 10: AI Coach

Goal: Explain detected build interactions without inventing game mechanics.

Work steps

- ☐ Reuse the Build Analyzer input
- ☐ Provide only verified synergies as facts
- ☐ Mark unknown combinations as unknown
- ☐ Create a rule-based summary fallback
- ☐ Structure responses into Pros, Risks, and Known Interactions
- ☐ Display the relevant game version

Definition of Done

- ☐ Every statement can be traced to a data record
- ☐ The coach does not invent missing synergies
- ☐ Empty or unknown builds are handled clearly

Result: A data-grounded explanation assistant as the final expansion stage.

5. Recommended Project Structure

```

frontend/
├── public/
│   ├── images/
│   │   ├── items/
│   │   └── synergies/
├── src/
│   ├── components/
│   │   ├── ItemCard.jsx
│   │   ├── SynergyCard.jsx
│   │   ├── SearchBar.jsx
│   │   └── SynergyGraph.jsx
│   ├── pages/
│   │   ├── HomePage.jsx
│   │   ├── ItemPage.jsx
│   │   ├── BuildPage.jsx
│   │   └── DataPage.jsx
│   ├── data/
│   │   └── items.json

```

```
| | └─ synergies.json
| └─ utils/
|   └─ itemHelpers.js
|   └─ synergyHelpers.js
| └─ App.jsx
| └─ App.css
| └─ main.jsx
└─ package.json
└─ README.md
```

6. Data Rules

- Item IDs are lowercase and use hyphens, for example sacred-heart.
- Image files use the same ID, for example sacred-heart.png.
- A synergy contains an items array and can reference two or more items.
- Descriptions are not invented. Unreviewed content uses verified: false.
- Game versions are stored because interactions may differ by version.
- Missing information is clearly shown as not documented.

7. Immediate Next Work Block

Current mini-goal

Finish React Router and open /item/godhead from the home page. Do not add images, a large data import, or the AI Coach during this work block.

- ☐ Check Git status and commit the current working state
- ☐ Create HomePage.jsx and ItemPage.jsx
- ☐ Change App.jsx to use Routes
- ☐ Verify BrowserRouter in main.jsx
- ☐ Render item names as links on HomePage
- ☐ Click Godhead and verify the URL
- ☐ Display synergies on ItemPage
- ☐ Test an item without a synergy
- ☐ Fix errors and create another commit

8. Quality Checklist Before Every Milestone

- ☐ No red error overlay in the browser
- ☐ No warning about missing React keys
- ☐ Home page loads after refresh
- ☐ A direct item URL loads after refresh
- ☐ Back navigation works
- ☐ Empty data does not crash the app
- ☐ Names and IDs match across JSON files
- ☐ Layout is readable on desktop and narrow screens

- ❑ Commit message describes one clear change
- ❑ README is updated after major architecture changes

9. Git Commit Examples

```
git commit -m "Add dynamic item pages"
```

```
git commit -m "Add multi-item synergy model"
```

```
git commit -m "Add item search and quality filter"
```

```
git commit -m "Add item image fallback"
```

```
git commit -m "Add interactive synergy graph"
```

```
git commit -m "Add build analyzer logic"
```